

TÀI LIỆU VẬN HÀNH · ĐỒ ÁN TỐT NGHIỆP

Hướng dẫn cài đặt và chạy dự án từ con số 0

Hệ thống quản lý nhà hàng tích hợp trợ lý AI (Text-to-SQL tiếng Việt). Tài liệu đi qua từng bước: chuẩn bị máy, cài công cụ, dựng CSDL, sinh dữ liệu mẫu, chạy backend và frontend, bật trợ lý AI, thanh toán QR qua SePay, chạy kiểm thử và xử lý sự cố.

Nội dung

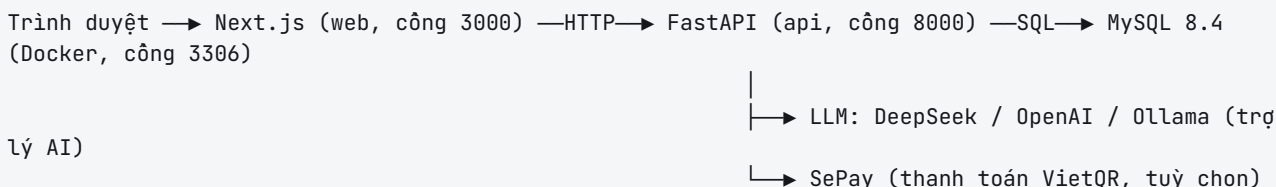
- | | |
|-------------------------------------|-----------------------------|
| 01 Tổng quan hệ thống | 10 Chạy backend (API) |
| 02 Chuẩn bị máy và cài công cụ | 11 Chạy frontend (web) |
| 03 Lấy mã nguồn | 12 Dùng thử các chức năng |
| 04 Cài thư viện (dependencies) | 13 Cấu hình trợ lý AI (LLM) |
| 05 Cấu hình biến môi trường | 14 Thanh toán QR qua SePay |
| 06 Khởi động MySQL bằng Docker | 15 Kiểm thử và đánh giá |
| 07 Tạo lược đồ CSDL (migration) | 16 Bảng lệnh thường dùng |
| 08 View và tài khoản chỉ-đọc cho AI | 17 Xử lý sự cố |
| 09 Sinh dữ liệu mẫu 12 tháng | |

Kho mã nguồn	github.com/nminhzzz/restaurant-management-ai
Nhánh	main
Cập nhật	26/09/2026
Hệ điều hành	macOS (chính) · Linux · Windows qua WSL2

1. Tổng quan hệ thống

Hệ thống gồm hai khối. **Khối nghiệp vụ** quản lý nhà hàng với 5 module: Danh mục, Bán hàng, Kho, Báo cáo, Cài đặt. **Khối trợ lý AI** cho phép hỏi dữ liệu kinh doanh bằng tiếng Việt; câu hỏi được chuyển thành SQL, kiểm duyệt rồi mới chạy.

1.1 Kiến trúc



1.2 Thành phần trong kho mã

THƯ MỤC	NỘI DUNG
<code>apps/api</code>	Backend Python 3.12 + FastAPI + SQLAlchemy 2 (async) + Alembic. Mỗi module nghiệp vụ nằm trong <code>src/app/modules/</code> .
<code>apps/web</code>	Frontend Next.js 16 (App Router) + React 19 + TypeScript + Tailwind 4.
<code>db/views</code>	Ba view phân quyền cho trợ lý AI (<code>vw_ai_quanly</code> , <code>vw_ai_thungan</code> , <code>vw_ai_kho</code>) và mẫu cấp quyền.
<code>scripts/seed</code>	Sinh dữ liệu mô phỏng 12 tháng (khoảng 20.000 order).
<code>scripts/apply_grants.sh</code>	Tạo view và 3 tài khoản CSDL chỉ-đọc cho trợ lý AI.
<code>data/eval</code>	Bộ 95 câu hỏi tiếng Việt – SQL chuẩn và harness đánh giá A/B/C.
<code>docs</code>	Báo cáo đồ án, sơ đồ UML, design system, spec và plan.
<code>Makefile</code>	Tập lệnh tắt: <code>make setup</code> , <code>make api</code> , <code>make gate</code> ... Gõ <code>make help</code> để xem hết.

1.3 Thứ tự tổng quát

Làm đúng thứ tự dưới đây; mỗi bước phụ thuộc bước trước:

1. Cài công cụ (mục 2) → lấy mã (mục 3) → cài thư viện (mục 4).
2. Tạo file `.env` (mục 5) → bật MySQL (mục 6) → tạo bảng (mục 7).
3. Tạo view + tài khoản chỉ-đọc cho AI (mục 8) → sinh dữ liệu mẫu (mục 9).
4. Chạy API (mục 10) và web (mục 11) ở hai cửa sổ Terminal riêng → mở trình duyệt.

Thời gian dự kiến

Lần đầu khoảng 30–45 phút (chủ yếu là tải Docker image, thư viện và sinh dữ liệu mẫu). Những lần sau chỉ cần 2 lệnh `make api` và `make web`.

2. Chuẩn bị máy và cài công cụ

2.1 Cấu hình tối thiểu

- RAM 8 GB (khuyến nghị 16 GB nếu chạy Ollama cục bộ), trống ≥ 10 GB ổ đĩa.
- Kết nối Internet cho lần cài đầu (tải thư viện, Docker image) và khi dùng LLM thương mại.

2.2 Danh sách công cụ

CÔNG CỤ	PHIÊN BẢN	DÙNG ĐỂ
Git	bất kỳ	Tải mã nguồn
Python	3.12 trở lên	Chạy backend
uv	mới nhất	Quản lý Python và thư viện backend
Node.js	20 trở lên	Chạy frontend
pnpm	9.15.0 (khóa trong <code>package.json</code>)	Cài thư viện frontend
Docker Desktop hoặc OrbStack	mới nhất	Chạy MySQL 8.4
make	có sẵn trên macOS/Linux	Chạy các lệnh tắt trong <code>Makefile</code>

2.3 Cài trên macOS

- 1 Cài **Homebrew** (trình quản lý gói) nếu máy chưa có. Mở ứng dụng **Terminal** và dán:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Cài xong, làm theo 2 dòng "Next steps" mà Homebrew in ra (thêm `brew` vào PATH), rồi đóng và mở lại Terminal.

- 2 Cài Git, uv, Node.js:

```
brew install git uv node
```

- 3 Bật pnpm đúng phiên bản bằng Corepack (đi kèm Node.js):

```
corepack enable
corepack prepare pnpm@9.15.0 --activate
```

- 4 Cài Python 3.12 qua uv (không đụng tới Python của hệ điều hành):

```
uv python install 3.12
```

- 5 Cài **Docker Desktop** (docker.com/products/docker-desktop) hoặc **OrbStack** (orbstack.dev – nhẹ hơn trên Mac). Mở ứng dụng lên một lần và chờ biểu tượng báo "running".

6 Kiểm tra lại toàn bộ. Mỗi lệnh phải in ra số phiên bản, không báo "command not found":

```
git --version
uv --version
uv run --python 3.12 python --version # Python 3.12.x
node --version                        # v20 trở lên
pnpm --version                        # 9.15.0
docker --version
docker compose version
make --version
```

2.4 Cài trên Windows (qua WSL2)

Makefile và script `apply_grants.sh` dùng bash, nên trên Windows nên chạy trong **WSL2 (Ubuntu)**:

1. Mở PowerShell quyền Administrator: `wsl --install -d Ubuntu`, khởi động lại máy, tạo tài khoản Ubuntu.
2. Cài Docker Desktop cho Windows, vào **Settings → Resources → WSL integration**, bật cho Ubuntu.
3. Trong cửa sổ Ubuntu: `sudo apt update && sudo apt install -y git make curl`, cài uv bằng `curl -Lsf https://astral.sh/uv/install.sh | sh`, cài Node 20 bằng `nvm` hoặc gói NodeSource, rồi làm tiếp bước 3-6 ở mục 2.3.
4. Để mã nguồn trong ổ Linux (ví dụ `~/projects`), không đặt trong `/mnt/c` – chạy sẽ chậm hơn nhiều.

Không dùng WSL?

Vẫn chạy được bằng cách gõ tay các lệnh mà Makefile gọi (xem mục 16), và dùng `scripts/apply_grants.ps1` thay cho `apply_grants.sh`.

3. Lấy mã nguồn

```
mkdir -p ~/projects && cd ~/projects
git clone https://github.com/nminhzzz/restaurant-management-ai.git
cd restaurant-management-ai
git checkout main
git pull
```

Từ đây trở đi, **mọi lệnh đều chạy ở thư mục gốc** `restaurant-management-ai`, trừ khi tài liệu ghi rõ `cd` vào thư mục khác.

4. Cài thư viện (dependencies)

```
make setup
```

Lệnh này làm hai việc:

- `cd apps/api && uv sync`: tạo môi trường ảo Python tại `apps/api/.venv` và cài FastAPI, SQLAlchemy, Alembic, pytest...
- `cd apps/web && pnpm install`: cài Next.js, React, Tailwind... vào `apps/web/node_modules`.

Thành công khi

Lệnh kết thúc không có dòng `error`; thấy thư mục `apps/api/.venv` và `apps/web/node_modules`.

Nếu pnpm báo sai phiên bản

Chạy lại `corepack prepare pnpm@09.15.0 --activate`. Nếu uv báo thiếu Python 3.12, chạy `uv python install 3.12`.

Tuỳ chọn cho người đóng góp mã: cài hook kiểm tra định dạng trước mỗi commit bằng `pipx install pre-commit && pre-commit install`.

5. Cấu hình biến môi trường

Tạo file `.env` ở thư mục gốc từ file mẫu `.env.example`:

```
make env
```

Lệnh chỉ tạo `.env` nếu chưa có, nên chạy lại nhiều lần cũng không sao. Mở `.env` bằng trình soạn thảo bất kỳ (VS Code: `code .env`). Với máy phát triển, **giữ nguyên giá trị mặc định là chạy được**; chỉ cần điền thêm phần AI (mục 8, 13) và SePay (mục 14) khi dùng tới.

Không bao giờ commit file .env

File này chứa mật khẩu và khoá API. Nó đã nằm trong `.gitignore`; chỉ `.env.example` (không có giá trị bí mật) mới được đưa lên Git.

5.1 Biến của backend (API)

BIẾN	MẶC ĐỊNH	Ý NGHĨA
ENVIRONMENT	local	Khác <code>local</code> thì bắt buộc đặt <code>JWT_SECRET</code> và <code>PAYMENT_WEBHOOK_SECRET</code> .
DATABASE_URL	mysql+asyncmy://restaurant:restaurant@localhost:3306/restaurant?charset=utf8mb4	Chuỗi kết nối MySQL cho nghiệp vụ. Khớp sẵn với Docker ở mục 6.
JWT_SECRET	chuỗi dev	Khoá ký token đăng nhập. Máy thật: đặt chuỗi ngẫu nhiên ≥ 32 ký tự (<code>openssl rand -hex 32</code>).
ACCESS_TOKEN_TTL_MINUTES	480	Thời gian sống của phiên đăng nhập (phút).
CORS_ORIGINS	["http://localhost:3000"]	Địa chỉ web được phép gọi API. Đối cổng web thì phải sửa ở đây (dạng JSON).
AI_PROVIDER	deepseek	deepseek · openai · ollama (mục 13).
AI_API_KEY	(trống)	Khoá API của nhà cung cấp LLM.
LLM_MODEL	deepseek-flash	Tên mô hình.

LLM_BASE_URL	(trống)	Để trống thì dùng địa chỉ mặc định của nhà cung cấp.
AI_OLLAMA_BASE_URL	http://localhost:11434	Địa chỉ Ollama khi AI_PROVIDER=ollama.
AI_READONLY_URL_MANAGER · _CASHIER · _WAREHOUSE	(trống)	Chuỗi kết nối của 3 tài khoản chỉ-đọc cho trợ lý AI (mục 8). Thiếu thì trợ lý không chạy được truy vấn.
AI_MAX_ROWS · AI_SQL_TIMEOUT_SECONDS · AI_MAX_SQL_ATTEMPTS	500 · 3 · 2	Giới hạn an toàn theo NFR-06. Không cần đổi.
PAYMENT_GATEWAY	simulator	simulator (giả lập) hoặc sepay (mục 14).
PAYMENT_WEBHOOK_SECRET	(trống)	Khoá HMAC của bộ giả lập QR.
SEPAY_*	(trống), prefix TT	Cấu hình SePay (mục 14).

5.2 Biến của MySQL (Docker)

MYSQL_ROOT_PASSWORD (mặc định restaurant-root), MYSQL_DATABASE (restaurant), MYSQL_USER / MYSQL_PASSWORD (restaurant), MYSQL_PORT (3306). Nếu đổi user/mật khẩu ở đây, nhớ đổi DATABASE_URL cho khớp.

5.3 Biến của frontend (web)

Web mặc định gọi API ở http://localhost:8000/api/v1. Chỉ khi API chạy ở địa chỉ khác mới cần tạo apps/web/.env.local:

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
NEXT_PUBLIC_API_PREFIX=/api/v1
```

6. Khởi động MySQL bằng Docker

- 1 Mở Docker Desktop / OrbStack và đợi nó chạy. Sau đó:

```
make db-up
```

Lần đầu Docker tải image mysql:8.4 (vài trăm MB). Container tên restaurant-db, dữ liệu nằm trong volume db-data nên tắt/bật lại không mất.

- 2 Chờ MySQL sẵn sàng (khoảng 20–40 giây lần đầu). Kiểm tra:

```
docker compose ps
```

Cột **STATUS** phải là Up ... (healthy). Nếu còn (health: starting) thì đợi thêm rồi chạy lại.

- 3 (Tuỳ chọn) vào thử MySQL bằng dòng lệnh:

```
docker compose exec db mysql -urestaurant -prestaurant restaurant -e "SELECT VERSION();"
```

Tắt MySQL: make db-down (dữ liệu vẫn giữ). Múi giờ container đặt sẵn Asia/Ho_Chi_Minh, bảng mã utf8mb4 để lưu tiếng Việt có dấu.

7. Tạo lược đồ CSDL (migration)

```
make migrate
```

Alembic chạy toàn bộ migration trong `apps/api/migrations/versions`, tạo 29 bảng nghiệp vụ theo báo cáo (`MON_AN`, `ORDER`, `HOA_DON`, `NGUYEN_LIEU` ...) cộng bảng đếm `DEM_ORDER`.

Thành công khi

Dòng cuối có dạng `Running upgrade ... -> ...` và không có `Traceback`. Kiểm tra: `docker compose exec db mysql -urestaurant -prestaurant restaurant -e "SHOW TABLES;"`.

Nếu báo "Can't connect to MySQL server"

MySQL chưa healthy (quay lại mục 6, bước 2) hoặc cổng 3306 đang bị một MySQL khác trên máy chiếm (xem mục 17).

8. View và tài khoản chỉ-đọc cho trợ lý AI

Trợ lý AI **không** đọc thẳng bảng nghiệp vụ. Mỗi vai trò chỉ thấy một view, và truy vấn bằng một tài khoản MySQL riêng chỉ có quyền `SELECT` trên đúng view đó (NFR-06):

VAI TRÒ	VIEW	TÀI KHOẢN MYSQL
Quản lý	<code>vw_ai_quanly</code>	<code>ai_manager</code>
Thu ngân	<code>vw_ai_thungan</code>	<code>ai_cashier</code>
Thủ kho	<code>vw_ai_kho</code>	<code>ai_warehouse</code>

- 1 Chọn 3 mật khẩu riêng (chỉ dùng chữ và số cho dễ đặt vào URL), đặt vào biến tạm của Terminal:

```
export AI_READONLY_PASSWORD_MANAGER='DoiMatKhouQuanLy1'
export AI_READONLY_PASSWORD_CASHIER='DoiMatKhouThuNgan1'
export AI_READONLY_PASSWORD_WAREHOUSE='DoiMatKhouKho1'
```

- 2 Chạy script. Script đọc 3 file view trong `db/views/`, tạo view trong CSDL, rồi tạo/đổi mật khẩu và cấp quyền cho 3 tài khoản:

```
./scripts/apply_grants.sh
```

Windows PowerShell: đặt `$env:AI_READONLY_PASSWORD_MANAGER="..."` (tương tự cho 2 biến kia) rồi `powershell - ExecutionPolicy Bypass -File scripts/apply_grants.ps1`. Nếu đổi `MYSQL_ROOT_PASSWORD`, export thêm biến đó trước khi chạy.

- 3 Điền 3 chuỗi kết nối vào `.env`, đúng mật khẩu vừa đặt:

```
AI_READONLY_URL_MANAGER=mysql+asyncmy://ai_manager:DoiMatKhouQuanLy1@localhost:3306/restaurant?charset=utf8mb4
AI_READONLY_URL_CASHIER=mysql+asyncmy://ai_cashier:DoiMatKhouThuNgan1@localhost:3306/restaurant?charset=utf8mb4
AI_READONLY_URL_WAREHOUSE=mysql+asyncmy://ai_warehouse:DoiMatKhouKho1@localhost:3306/restaurant?charset=utf8mb4
```

Mật khẩu có ký tự đặc biệt?

Các ký tự `@ : / ? # %` phải mã hoá URL trong chuỗi kết nối (ví dụ `@` thành `%40`). Dùng chữ và số là cách đơn giản nhất.

Chạy lại sau mỗi lần tạo lại CSDL

View và tài khoản nằm trong CSDL. Nếu xoá volume và chạy lại migration (mục 9.4), phải chạy lại `apply_grants.sh`.

9. Sinh dữ liệu mẫu 12 tháng

CSDL mới tạo chưa có tài khoản nào để đăng nhập. Script seed tạo toàn bộ dữ liệu để dùng thử và làm báo cáo: vai trò, cấu hình, khoảng 70 món, nguyên liệu, công thức, 24 bàn, 3 tài khoản mẫu và khoảng 20.000 order trong 12 tháng (có phiếu nhập, xuất kho, thanh toán, trừ kho FIFO, Business Date 06:00–06:00).

9.1 Chạy seed

```
make seed
```

Mất vài phút tùy máy. Kết thúc sẽ in dạng:

```
[seed] danh mục: 71 món · ... nguyên liệu · 24 bàn
[seed] vận hành: ... order · ... phiếu nhập · ... phiếu xuất
[seed] xong lúc 2026-...
```

9.2 Tuỳ chỉnh lượng dữ liệu

Muốn chạy nhanh để thử (ít order hơn):

```
cd apps/api
uv run python ../../scripts/seed/generate.py --orders 3000 --months 3 --seed 42
cd ../../
```

`--orders`: số order (mặc định 20.000) · `--months`: số tháng lùi về trước (12) · `--seed`: hạt giống ngẫu nhiên, cùng số thì ra cùng dữ liệu (42).

9.3 Tài khoản mẫu

TÊN ĐĂNG NHẬP	MẬT KHẨU	VAI TRÒ	THẤY CÁC MODULE
<code>seed_manager</code>	<code>matkhau123</code>	Quản lý	Tất cả 6 module
<code>seed_cashier</code>	<code>matkhau123</code>	Thu ngân	Bán hàng, Trợ lý AI...
<code>seed_warehouse</code>	<code>matkhau123</code>	Thủ kho	Kho, Trợ lý AI...

Chỉ dùng cho máy phát triển

Đây là mật khẩu mẫu công khai. Khi triển khai thật, đăng nhập bằng `seed_manager`, vào **Cài đặt → Tài khoản** tạo tài khoản mới, rồi khoá các tài khoản mẫu.

9.4 Làm lại CSDL từ đầu

Seed chỉ chạy trên CSDL sạch; chạy lần hai sẽ báo *"CSDL đã có dữ liệu danh mục"*. Để làm lại:

Xoá toàn bộ dữ liệu

Các lệnh dưới đây xoá vĩnh viễn mọi dữ liệu trong MySQL cục bộ. Chỉ làm trên máy phát triển.

```
docker compose down -v      # tắt MySQL và xoá volume dữ liệu
make db-up                  # đợi healthy (docker compose ps)
make migrate
./scripts/apply_grants.sh    # nhớ export lại 3 mật khẩu ở mục 8
make seed
```

10. Chạy backend (API)

Mở một cửa sổ Terminal riêng cho API (để nó chạy suốt):

```
make api
```

Tương đương `cd apps/api && uv run uvicorn app.main:app --reload --app-dir src --port 8000`. Chế độ `--reload` tự khởi động lại khi sửa mã.

Thành công khi

Terminal in `Application startup complete` và `Uvicorn running on http://127.0.0.1:8000`. Mở <http://localhost:8000/docs> để xem toàn bộ API (Swagger) và gọi thử từng endpoint.

Dừng API: bấm `Ctrl` + `C` trong cửa sổ đó. Sửa `.env` xong phải dừng rồi chạy lại để API đọc giá trị mới.

11. Chạy frontend (web)

Mở cửa sổ Terminal thứ hai:

```
make web
```

Tương đương `cd apps/web && pnpm dev`. Đợi dòng `Ready` rồi mở <http://localhost:3000>.

- 1 Trang đăng nhập hiện ra. Nhập `seed_manager` / `matkhau123`, bấm **Đăng nhập**.
- 2 Sidebar tối bên trái liệt kê các module theo vai trò. Góc dưới sidebar là tên người dùng, nút đổi mật khẩu và đăng xuất.

Muốn chạy web ở cổng khác?

`cd apps/web && pnpm exec next dev -p 3100`, đồng thời sửa `CORS_ORIGINS=["http://localhost:3100"]` trong `.env` và khởi động lại API – nếu không, trình duyệt sẽ báo lỗi CORS và mọi trang đều "Không tải được dữ liệu".

11.1 Chạy bản build production (tuỳ chọn)

```
cd apps/web
pnpm build
pnpm start # mặc định cổng 3000
```

12. Dùng thử các chức năng

12.1 Bán hàng – 3 bước (đăng nhập `seed_cashier`)

1. **Chọn bàn.** Lưới bàn hiện trạng thái: *Trống* (xanh), *Có khách* (đen, kèm tạm tính, số món, giờ mở), *Đã đặt* (xám). Lọc theo trạng thái ở thanh phía trên; ô "Tra mã order" để tìm order cũ.
2. Bấm **bàn trống** → sang bước **Gọi món** với order mới. Bấm **bàn có khách** → khung bên phải hiện tóm tắt với nút **Gọi thêm** và **Thanh toán**. Đơn mang về: nút **Đơn mới** ở khung "Mang về".
3. **Gọi món.** Mỗi ô món hiện giá; bấm để thêm, dùng /[-] để đổi số lượng, thêm ghi chú cho bếp. Món chưa có giá hiện "Chưa có giá" và không thêm được. Giỏ hàng chia "Món mới" và "Đã gửi bếp", tổng tiền cập nhật ngay.
4. Bấm **Gửi bếp** (quay về sơ đồ bàn) hoặc **Gửi bếp & thanh toán** (sang bước 3).
5. **Thanh toán.** Chọn **Tiền mặt**: nhập tiền khách đưa hoặc bấm nút nhanh ("Vừa đủ", 50.000, 100.000...), màn hình tính tiền thối, bấm **Xác nhận đã thu**. Hoặc chọn **QR chuyển khoản** (mục 14).
6. Màn **Hoàn tất** hiện số hoá đơn, mã order, phương thức, tổng tiền, tiền thối, nút **In hoá đơn** và **Về sơ đồ bàn**. Bàn tự chuyển về *Trống*.

Vị trí đang làm được lưu trên thanh địa chỉ (`/sales?step=pay&order=...`), nên tải lại trang không mất chỗ.

12.2 Các module khác (đăng nhập `seed_manager`)

MODULE	THỬ GÌ
Danh mục	Món, nhóm món, nguyên liệu, nhà cung cấp, bàn; đặt giá có hiệu lực theo ngày, sửa công thức món.
Kho	Tồn kho và cảnh báo dưới mức tối thiểu, phiếu nhập, phiếu xuất, kiểm kê.
Báo cáo	Doanh thu theo ngày/tuần/tháng/năm, món bán chạy, khung giờ, lợi nhuận, so sánh kỳ, order hủy.
Cài đặt	Tài khoản và vai trò, cấu hình nhà hàng, nhật ký hệ thống, sao lưu.

12.3 Trợ lý AI

- **Màn đầy đủ** (mục "Trợ lý AI"): cột lịch sử trò chuyện bên trái, màn chào với 4 câu hỏi gợi ý theo vai trò. Câu trả lời gồm câu kết luận, các ý chính, tab **Biểu đồ / Bảng / SQL**, ghi chú phạm vi dữ liệu, nút **Sao chép**, **Hỏi lại** và các câu **Hỏi tiếp**.
- **Ô chat nổi** ở góc dưới phải mọi màn (trừ Bán hàng và màn Trợ lý). Mở/đóng bằng `Ctrl + K` (Mac: `⌘ + K`), `Esc` để thu nhỏ; nút "Mở toàn màn hình" chuyển sang màn đầy đủ đúng phiên đang hỏi.
- Trợ lý **chỉ đọc** dữ liệu và chỉ trong phạm vi vai trò. Cần cấu hình LLM (mục 13) và tài khoản chỉ-đọc (mục 8) mới trả lời được.

13. Cấu hình trợ lý AI (LLM)

Chọn **một** trong ba cách, sửa `.env`, rồi khởi động lại API.

13.1 DeepSeek (mặc định, rẻ, nhanh)

1. Tạo tài khoản tại platform.deepseek.com, nạp tiền tối thiểu, vào **API Keys** → tạo khoá.
2. Trong `.env`:

```
AI_PROVIDER=deepseek
AI_API_KEY=sk-...khóa-của-bạn...
LLM_MODEL=deepseek-flash # mặc định của dự án
```

Để trống `LLM_BASE_URL` (dùng `https://api.deepseek.com`). `LLM_REASONING_EFFORT=low` giữ mỗi câu hỏi trong ngân sách 8 giây (NFR-02).

13.2 OpenAI

```
AI_PROVIDER=openai
AI_API_KEY=sk-...
LLM_MODEL=gpt-4o-mini # ví dụ; chọn mô hình còn hỗ trợ trong tài khoản của bạn
```

13.3 Ollama (chạy mô hình ngay trên máy, không cần khoá)

1. Cài Ollama (ollama.com) và mở ứng dụng.
2. Tải một mô hình giỏi SQL, ví dụ: `ollama pull qwen2.5-coder:7b` (khoảng 4–5 GB).
3. Trong `.env`:

```
AI_PROVIDER=ollama
LLM_MODEL=qwen2.5-coder:7b
AI_OLLAMA_BASE_URL=http://localhost:11434
```

Kiểm tra nhanh

Đăng nhập `seed_manager` → Trợ lý AI → bấm gợi ý "Doanh thu 7 ngày gần nhất theo từng ngày?". Có bảng và biểu đồ là đã thông. Nếu báo lỗi, xem API log trong cửa sổ `make api` và mục 17.

Các giới hạn an toàn luôn bật: mọi câu SQL do mô hình sinh ra phải qua bộ kiểm duyệt `validate_sql` (chỉ `SELECT` trên đúng view), tối đa 500 dòng, 3 giây mỗi truy vấn, 2 lần sinh SQL mỗi câu hỏi, hạn mức `AI_DAILY_QUESTION_QUOTA` câu/ngày.

14. Thanh toán QR qua SePay

Có hai chế độ, chọn bằng `PAYMENT_GATEWAY`:

- `simulator` (mặc định): cổng giả lập dùng cho kiểm thử tự động. Trên màn thanh toán vẫn tạo được giao dịch QR có đếm ngược 10 phút; khi demo, dùng **Tiền mặt** hoặc luồng "Khách báo đã chuyển khoản → Chờ đối soát → Quản lý xác nhận".
- `sepay`: mã VietQR chuyển khoản thật qua SePay ở **Test mode** (không mất tiền), hệ thống tự xác nhận khi SePay gọi webhook.

14.1 Chuẩn bị SePay Test mode

- 1 Đăng ký tài khoản tại **my.sepay.vn**, bật **Test mode** (chế độ thử nghiệm) trên thanh bên, tạo một **tài khoản ngân hàng Test mode**. Ghi lại số tài khoản, mã ngân hàng và tên chủ tài khoản.
- 2 Trong mục **Cấu trúc mã thanh toán**, khai báo tiền tố **TT** (phải trùng `SEPAY_PAYMENT_PREFIX`). Mỗi giao dịch có nội dung dạng `TT388`.
- 3 Mở đường hầm HTTPS để SePay gọi được máy của bạn. Cài cloudflared (`brew install cloudflared`) rồi chạy trong cửa sổ Terminal thứ ba:

```
cloudflared tunnel --url http://localhost:8000
```

Cloudflared in ra một địa chỉ dạng `https://ten-ngau-nhien.trycloudflare.com`.

- 4 Trong SePay, thêm **webhook**: URL `https://ten-ngau-nhien.trycloudflare.com/api/v1/sales/webhooks/sepay`, kiểu xác thực **API Key**, đặt một chuỗi khoá ngẫu nhiên (ví dụ `openssl rand -hex 24`). (Tùy chọn) tạo **API Token** để dùng nút "Kiểm tra lại".

14.2 Cấu hình `.env`

```
PAYMENT_GATEWAY=sepay
SEPAY_BANK_ACCOUNT=0123499999          # số tài khoản Test mode
SEPAY_BANK_CODE=MBBank                 # mã ngân hàng theo SePay
SEPAY_ACCOUNT_NAME=NHA HANG DEMO
SEPAY_WEBHOOK_API_KEY=chuo-i-khoa-o-buoc-4
SEPAY_API_TOKEN=token-tuy-chon
SEPAY_PAYMENT_PREFIX=TT
```

Khởi động lại API. Thiếu số tài khoản, mã ngân hàng hoặc API key thì API sẽ báo lỗi ngay khi khởi động – đây là cố ý để không chạy với cấu hình thiếu.

14.3 Demo một giao dịch

1. Bán hàng → chọn bàn có khách → **Thanh toán** → **QR chuyển khoản** → **Tạo mã QR**. Màn hình hiện mã VietQR, số tiền, ngân hàng, số tài khoản, nội dung chuyển khoản (có nút sao chép) và đếm ngược 10 phút.
2. Trong trang SePay, dùng **Giả lập giao dịch** với **đúng số tiền** và **đúng nội dung** (ví dụ `TT388`).
3. Trong vòng vài giây màn thu ngân tự chuyển sang **Hoàn tất**. Không có tunnel thì bấm **Kiểm tra lại** (cần `SEPAY_API_TOKEN`).

TÌNH HUỐNG	HỆ THỐNG XỬ LÝ
Chuyển đúng số tiền, còn hạn	Giao dịch "Thành công", tạo hoá đơn, trả bàn.
SePay gửi lại cùng một giao dịch	Bỏ qua, không tạo hoá đơn thứ hai.
Sai số tiền hoặc chuyển sau khi QR hết hạn	Chuyển "Chờ đối soát"; quản lý xác nhận thủ công.
Tiền vào order đã trả bằng tiền mặt	Ghi nhật ký cho người kiểm tra; không đổi order.
Sai API key	Từ chối (401) và ghi nhật ký.

Bảo mật

Không đưa `SEPAY_WEBHOOK_API_KEY`, `SEPAY_API_TOKEN` vào Git hay ảnh chụp màn hình. Khi `PAYMENT_GATEWAY=sepay`, webhook giả lập tự tắt để không ai giả mạo xác nhận thanh toán qua tunnel.

15. Kiểm thử và đánh giá

15.1 Cổng kiểm tra đầy đủ (chạy trước khi commit)

```
make gate
```

Chạy lần lượt: kiểm tra định dạng (ruff format, prettier) → lint (ruff, ESLint) → kiểu (mypy, tsc) → test (pytest, vitest) → build web. Mọi bước phải xanh. Bộ test dùng CSDL SQLite trong bộ nhớ và LLM giả, **không cần** MySQL, khoá API hay mạng.

```
make fmt          # tự sửa định dạng
make test-api     # chỉ backend: cd apps/api && uv run pytest
make test-web     # chỉ frontend: cd apps/web && pnpm test
cd apps/api && uv run pytest tests/modules/test_sales_payments.py -q # một file
```

15.2 Kiểm thử trên MySQL thật (tuỳ chọn)

```
make gate-full    # gate + test integration nếu máy đủ RAM/đĩa và MySQL đang chạy
```

15.3 Đánh giá trợ lý AI (thực nghiệm A/B/C)

Cần MySQL có dữ liệu seed, tài khoản chỉ-đọc và LLM đã cấu hình:

```
cd apps/api
uv run python ../../data/eval/harness.py --configs A,B,C --out ../../docs/eval-results.json
```

Chạy 95 câu hỏi trong `data/eval/questions.jsonl`, ghi kết quả vào `docs/eval-results.json`. Lần chạy tốn phí LLM thật.

16. Bảng lệnh thường dùng

LỆNH MAKE	LỆNH GỐC TƯƠNG ĐƯƠNG	VIỆC
<code>make help</code>	—	Liệt kê mọi lệnh
<code>make setup</code>	<code>uv sync</code> <code>(api) · pnpm install</code> <code>(web)</code>	Cài thư viện
<code>make env</code>	<code>cp .env.example .env</code>	Tạo .env nếu chưa có
<code>make db-up</code> / <code>db-down</code>	<code>docker compose up -d db</code> / <code>down</code>	Bật / tắt MySQL
<code>make migrate</code>	<code>uv run alembic upgrade head</code>	Tạo / nâng cấp bảng
<code>make seed</code>	<code>uv run python ../../scripts/seed/generate.py</code>	Sinh dữ liệu mẫu
<code>make api</code>	<code>uv run uvicorn app.main:app --reload --app-dir src --port 8000</code>	Chạy backend

<code>make web</code>	<code>pnpm dev</code>	Chạy frontend
<code>make gate</code>	<code>fmt-check · lint · typecheck · test · build</code>	Kiểm tra toàn bộ
<code>make build</code>	<code>pnpm build</code>	Build web production
<code>make clean</code>	xoá <code>.next</code> , cache pytest/mypy/ruff	Dọn cache build

16.1 Hằng ngày chỉ cần

```
make db-up      # nếu MySQL chưa chạy
make api       # cửa số 1
make web       # cửa số 2 → http://localhost:3000
```

17. Xử lý sự cố

HIỆN TƯỢNG	NGUYÊN NHÂN VÀ CÁCH XỬ LÝ
<code>Cannot connect to the Docker daemon</code>	Docker Desktop / OrbStack chưa mở. Mở ứng dụng, đợi "running", chạy lại <code>make db-up</code> .
<code>port is already allocated</code> (3306)	Máy đã có MySQL khác chạy. Tắt nó (<code>brew services stop mysql</code>) hoặc đặt <code>MYSQL_PORT=3307</code> trong <code>.env</code> và sửa cổng trong <code>DATABASE_URL</code> , <code>AI_READONLY_URL_*</code> .
<code>Address already in use</code> (8000 hoặc 3000)	Tìm tiến trình: <code>lsof -iTCP:8000 -sTCP:LISTEN</code> , rồi <code>kill <PID></code> . Hoặc chạy ở cổng khác (nhớ sửa <code>CORS_ORIGINS</code>).
<code>make migrate:</code> <code>Can't connect to MySQL</code>	MySQL chưa healthy – <code>docker compose ps</code> , đợi thêm. Kiểm tra user/mật khẩu trong <code>DATABASE_URL</code> khớp <code>MYSQL_USER</code> / <code>MYSQL_PASSWORD</code> .
Seed báo "CSDL đã có dữ liệu danh mục"	Seed chỉ chạy trên CSDL sạch. Làm lại theo mục 9.4.
Đăng nhập báo sai tài khoản	Chưa chạy seed (mục 9), hoặc gõ sai mật khẩu <code>matkhu123</code> .
Trang web hiện "Không tải được dữ liệu"	API chưa chạy (xem cửa sổ <code>make api</code>), hoặc lỗi CORS do web không chạy ở cổng 3000 – mở DevTools (<code>F12</code>) → Console; sửa <code>CORS_ORIGINS</code> rồi khởi động lại API.
Trợ lý AI báo lỗi hoặc "Không thể tạo truy vấn an toàn"	Thiếu <code>AI_API_KEY</code> / sai <code>LLM_MODEL</code> ; thiếu <code>AI_READONLY_URL_*</code> hoặc chưa chạy <code>apply_grants.sh</code> ; Ollama chưa mở. Xem log ở cửa sổ API. Câu hỏi ngoài phạm vi vai trò bị từ chối là đúng thiết kế.
Trợ lý trả lời chậm rồi "phản hồi quá lâu"	Ngân sách mỗi câu là 8 giây. Dùng mô hình nhanh hơn, hoặc với Ollama chọn mô hình nhỏ hơn.
QR SePay không tự xác nhận	Tunnel đã tắt/đổi địa chỉ (cập nhật lại URL webhook trong SePay); sai API key; nội dung chuyển khoản sai tiền tố; số tiền khác tổng order (sẽ vào "Chờ đối soát"). Dùng nút "Kiểm tra lại" nếu có <code>SEPAY_API_TOKEN</code> .
API không khởi động sau khi bật SePay	Thiếu <code>SEPAY_BANK_ACCOUNT</code> , <code>SEPAY_BANK_CODE</code> hoặc <code>SEPAY_WEBHOOK_API_KEY</code> ; đọc dòng lỗi đầu tiên trong log.
<code>pnpm: command not found</code> / sai phiên bản	<code>corepack enable && corepack prepare pnpm@9.15.0 --activate</code> .

uv không tìm thấy Python 3.12

`uv python install 3.12` rồi `make setup`.

`make gate` đở ở bước format

Chạy `make fmt` rồi chạy lại gate.

Web cũ vẫn hiện giao diện cũ sau khi
`git pull`

Dùng `make web`, chạy `make setup` (thư viện mới), `make migrate` (bảng mới), rồi chạy lại API và web. Nếu vẫn lỗi: `make clean`.

17.1 Khi cần báo lỗi cho nhóm

Gửi kèm: lệnh vừa chạy, 20 dòng cuối trong Terminal, đầu ra của `git log --oneline -1` và `docker compose ps`. **Xoá mọi khoá API, mật khẩu** khỏi nội dung trước khi gửi.

Tóm tắt lần chạy đầu tiên

`make setup` → `make env` → `make db-up` → `make migrate` → `./scripts/apply_grants.sh` (sau khi export 3 mật khẩu, rồi điền `AI_READONLY_URL_*` vào `.env`) → `make seed` → `make api` (cửa sổ 1) → `make web` (cửa sổ 2) → mở `http://localhost:3000`, đăng nhập `seed_manager` / `matkhu123`.